

Usuário:

no discord, qual a diferença entre banir e expulsar um membro de um servidor?

IA:

Estou buscando essa informação. A distinção pode ser sutil dependendo da configuração do servidor.

No Discord, a principal diferença entre as duas ações reside na **permanência** da punição e na facilidade com que o membro pode retornar ao servidor.

Abaixo, apresento um comparativo direto para facilitar o entendimento:

Aqui estão os detalhes adicionais sobre cada ação:

Expulsar (Kick)

Quando você expulsa alguém, o usuário é apenas "colocado para fora". Ele não perde o acesso ao servidor de forma definitiva; se ele encontrar um link de convite válido ou se o servidor for aberto, ele pode entrar novamente no mesmo instante. É uma medida utilizada geralmente como um "aviso final" ou para organizar o servidor removendo contas inativas.

Banir (Ban)

O banimento é a punição mais severa. Quando um membro é banido:

- A conta dele é removida do servidor.
- O **endereço de IP** dele é registrado na lista de banimentos, o que dificulta que ele crie uma conta secundária para tentar burlar a punição.
- Ele só conseguirá retornar se um administrador for até as **Configurações do Servidor > Banimentos** e remover o nome dele da lista manualmente.

Resumindo

- **Expulsar:** É uma remoção simples. O usuário pode voltar se tiver um convite.

- **Banir:** É uma proibição total. O usuário fica bloqueado (por conta e IP) até que a punição seja revogada.

Próximos passos:

- Se você for moderador, lembre-se de que ao banir, você pode escolher se deseja **deletar o histórico de mensagens** do usuário (das últimas 24 horas ou 7 dias).
- Caso tenha banido alguém por engano, você pode desfazer a ação na aba de "Banimentos" nas configurações do servidor.

Usuário:

https://hexdocs.pm/nostrum/Nostrum.Api.Guild.html#ban_member/4

Use o webcrawl para me ajudar: Me ajude na função `ban_member`. me explique os argumentos dessa função

IA:

Estou analisando a página. A documentação de APIs costuma ser bem detalhada sobre os parâmetros.

A função `ban_member/4` da biblioteca **Nostrum** (usada para criar bots de Discord em Elixir) é utilizada para banir um membro de um servidor específico.

Abaixo, explico detalhadamente cada um dos quatro argumentos aceitos por essa função, conforme a documentação técnica:

Definição da Função

```
```elixir
ban_member(guild_id, user_id, days_to_delete, reason \\ nil)
```
```

Argumentos

1. `guild_id` (Nostrum.Struct.Guild.id()):

- É o ID único (Snowflake) do servidor (guild) de onde o usuário será banido.
- Geralmente é um número inteiro grande.

2. ``user_id`` (Nostrum.Struct.User.id()):

- É o ID único do usuário que você deseja banir.
- Importante notar que o usuário não precisa estar necessariamente no servidor no momento do banimento (banimento preventivo).

3. ``days_to_delete`` (integer()):

- Este argumento define a quantidade de dias de **histórico de mensagens** desse usuário que o Discord deve apagar automaticamente.
- O valor deve ser um número inteiro, geralmente entre 0 e 7 dias. Se você não quiser deletar nenhuma mensagem, use 0.

4. ``reason`` (Nostrum.Struct.Guild.AuditLogEntry.reason()):

- **Opcional** (valor padrão é nil).
- É uma string que descreve o motivo do banimento.
- Este motivo aparecerá nos **Logs de Auditoria** do servidor no Discord, o que é fundamental para a transparência da moderação.

Exemplo de Uso

Se você quiser banir o usuário 123456789, no servidor 987654321, deletando as mensagens dele dos últimos 7 dias por "spam persistente", o código seria:

```
```elixir
```

```
Nostrum.Api.Guild.ban_member(987654321, 123456789, 7, "Spam persistente nos canais de texto")
```

```
```
```

Resumo do Retorno

- ``{:ok}``: O banimento foi executado com sucesso.
- ``{:error, reason}``: Ocorreu uma falha (ex: o bot não tem permissão de "Banir Membros" ou o ID fornecido é inválido).

Próximos passos:

- Certifique-se de que o seu bot tenha a permissão BAN_MEMBERS no servidor para que esta função funcione.
- Se precisar apenas remover o usuário sem impedir a volta dele, utilize a função kick_member/3.

Usuário:

mas days to delete pode ser 0 por padrão quando chamada?

IA:

Estou verificando isso. O comportamento padrão de parâmetros em chamadas de função pode variar bastante.

Sim, o argumento days_to_delete pode ser 0 por padrão, mas é importante entender como o **Nostrum** (e a API do Discord) lida com isso.

Na implementação da função ban_member/4 no Nostrum, esse argumento **não possui um valor padrão definido na assinatura da função** (diferente do reason, que é \ nil). Isso significa que, ao chamar a função, você é **obrigado** a passar um valor para ele.

Como funciona na prática:

Se você deseja banir um usuário sem apagar nenhuma mensagem dele, você deve passar explicitamente o valor 0.

Exemplo:

```
```elixir
```

### Forma CORRETA (passando 0 explicitamente)

```
Nostrum.Api.Guild.ban_member(guild_id, user_id, 0)
```

**Isso causará um ERRO de compilação/execução (faltando argumento)**

```
Nostrum.Api.Guild.ban_member(guild_id, user_id)
```

```
...
```

## Por que não tem padrão?

O Discord exige que a API receba uma instrução sobre o que fazer com as mensagens. No Nostrum, os desenvolvedores optaram por não assumir o 0 como padrão para garantir que o programador decida conscientemente se quer ou não limpar o histórico do usuário no momento do banimento.

## Resumo Técnico

- **É obrigatório?** Sim, você deve passar o número.
- **Pode ser 0?** Sim, use 0 para manter todas as mensagens do usuário intactas.
- **Qual o limite?** O Discord aceita valores de 0 a 7 (algumas versões mais recentes da API suportam até 7 dias, mas o comportamento padrão seguro é nesta faixa).

## Sugestão de Próximo Passo:

Se você estiver criando um comando de moderação genérico, uma boa prática em Elixir é criar uma função auxiliar no seu bot que já defina esse padrão:

```
```elixir

def meu_ban(guild_id, user_id, days \\ 0, reason \\ nil) do

  Nostrum.Api.Guild.ban_member(guild_id, user_id, days, reason)

end

```
```

## Usuário:

```
Api.Guild.ban_member(guild_id, member.user_id, days_to_delete: 0)
```

está certo assim?

## IA:

Estou verificando isso. A sintaxe da chamada de função parece estar correta.

Quase certo. No Elixir, a sintaxe de **Keyword Lists** (como `days_to_delete: 0`) só é utilizada para argumentos que são passados como uma lista de opções no final da função.

Como vimos na documentação da **Nostrum**, o argumento `days_to_delete` é um argumento **posicional** obrigatório (o terceiro da função), e não uma opção nomeada.

O código correto deve ser:

```
```elixir
```

O terceiro argumento deve ser apenas o número 0

```
Nostrum.Api.Guild.ban_member(guild_id, member.user_id, 0)
```

```
```
```

Se você quiser incluir o motivo (quarto argumento), ficaria assim:

```
```elixir
```

```
Nostrum.Api.Guild.ban_member(guild_id, member.user_id, 0, "Motivo do banimento")
```

```
```
```

## Por que a sua versão daria erro?

Ao escrever `days_to_delete: 0`, você está passando uma **Tupla** `{:days_to_delete, 0}` dentro de uma lista. A função `ban_member/4` espera um **Inteiro** no terceiro parâmetro, e não uma lista ou tupla, o que resultaria em um erro de `FunctionClauseError` ou erro de tipo.

## Resumindo

1. **guild\_id**: 1º argumento (ID do servidor).
2. **user\_id**: 2º argumento (ID do usuário).
3. **0**: 3º argumento (Quantidade de dias de mensagens para apagar).

4. **reason**: 4º argumento (String opcional).

### Próximos passos:

- Verifique se o `member.user_id` realmente contém o ID (inteiro) e não a struct completa do usuário.
- Se o seu bot precisar apagar mensagens, você pode trocar o 0 por qualquer valor até 7.

## Usuário:

qwen, temos o seguinte problema:

```
defp perform_bans(guild_id, selected) do
```

```
 Enum.each(selected, &ban_member(guild_id, &1))
```

```
 :ok
```

```
end
```

```
defp ban_member(guild_id, member) do
```

```
 case Api.Guild.ban_member(guild_id, member.user_id, 0, "roleta russa do davi britto") do
```

```
 {:ok, _} ->
```

```
 IO.inspect("Banned user #{member.user_id} (#{member.user.username})")
```

```
 {:error, reason} ->
```

```
 IO.inspect("Failed to ban #{member.user_id}: #{inspect(reason)}")
```

```
 end
```

```
end
```

na função *ban\_member* pode ser retornado `:ok` e `:error` mas na *perform\_bans* ela sempre retornará `:ok` e como consequencia na função *roleta\_russa*, dará um falso `:ok` de que tudo realizado com sucesso

## IA:

Estou verificando a lógica do código. A forma como os retornos são tratados pode causar um falso positivo.

O problema ocorre porque o Enum.each/2 é uma função de "efeito colateral": ela percorre a lista, executa a ação, mas sempre retorna o átomo :ok, independentemente do que aconteça dentro da função anônima.

Para que a função perform\_bans reflita o sucesso ou falha real, você deve mudar a lógica para verificar se houve algum erro durante o processo.

Aqui estão as duas melhores formas de resolver isso em Elixir:

## 1. Abordagem "Tudo ou Nada" (Fail Fast)

Se você quer que a operação pare e retorne erro assim que o primeiro banimento falhar, use o Enum.reduce\_while/3:

```
``elixir

defp perform_bans(guild_id, selected) do

 Enum.reduce_while(selected, :ok, fn member, _acc ->

 case ban_member(guild_id, member) do

 :ok -> {:cont, :ok}

 {:error, reason} -> {:halt, {:error, reason}}

 end

 end)

end

defp ban_member(guild_id, member) do
```

## Ajustado para retornar :ok puro conforme o padrão da API do Nostrum

```
case Api.Guild.ban_member(guild_id, member.user_id, 0, "roleta russa do davi britto") do

 {:ok} ->

 IO.inspect("Banned user #{member.user_id}")

 :ok

end
```



```

{:error, reason} ->

IO.inspect("Failed to ban #{member.user_id}: #{inspect(reason)}")

{:error, reason}

end

end

...

```

## 2. Abordagem Coletiva (Relatório de Erros)

Se você quer tentar banir todos e apenas saber se **algum** falhou ao final, use Enum.map/2 e verifique os resultados:

```

```elixir

defp perform_bans(guild_id, selected) do

  results = Enum.map(selected, &ban_member(guild_id, &1))

  if Enum.any?(results, fn res -> match?({:error, _}, res) end) do

    {:error, "Alguns usuários não puderam ser banidos"}

  else

    :ok

  end

end

...

```

O que mudou:

1. **Retorno da `ban\_member`**: Ajustamos para ela retornar apenas :ok (ou o que você desejar) em vez de apenas imprimir no console. Note que a Api.Guild.ban_member do Nostrum geralmente retorna {:ok} (uma tupla de um elemento) ou {:error, reason}.
2. **Fluxo da `perform\_bans`**: Agora ela consome o retorno de cada banimento individual. Se houver um erro, ele é propagado para a função roleta_russa.

Resumindo

- O Enum.each ignora o valor de retorno da função interna.
- Use Enum.reduce_while para interromper no primeiro erro.
- Use Enum.map + Enum.any? para processar todos e validar o resultado global.

Sugestão de próximo passo:

Verifique na função roleta_russa se você está usando o operador with ou um case para tratar esse novo retorno da perform_bans. Posso te ajudar a estruturar esse tratamento se desejar.

Usuário:

explique esse erro:

```
22:43:10.872 [error] State machine 'Elixir.Nostrum.Api.Ratelimiter' terminating
```

```
  Last event = {internal,{run,#{params => [],
```

```
  body =>
```

```
  #{embeds =>
```

```
  [{timestamp => nil,type => nil,
```

```
  description =>
```

```
  #{response =>
```

```
  #{code => 50013,
```

```
  message =>
```

```
  >},
```

```
  '__struct__' =>
```

```
  'Elixir.Nostrum.Error.ApiError',
```

```
  '__exception__' => true,
```

```
  status_code => 403},
```

```
  title =>
```

```
  <<226,157,140,32,69,114,114,
```

```
111,32,110,97,32,82,111,
108,101,116,97,32,82,117,
115,115,97>>,
author => nil,fields => nil,
image => nil,
'__struct__' =>
'Elixir.Nostrum.Struct.Embed',
color => 16711680,url => nil,
provider => nil,video => nil,
footer => nil,
thumbnail => nil}}},
headers =>
[>,
>]],
method => post,
route =>
>},
>,
{,
#Ref}}}
```

```

When server state = {connected,#{running=>#{},remaining_in_window=>50,
outstanding=>
#{>=>
{initial,{{}},
>=>
{999,{{}},

```

> =>

{9,{[],[]}},

wrapped_token =>

#Fun,

conn => ,inflight => #{}}}

Reason for termination = error:#{value =>

#{response =>

#{code => 50013,

message =>

>},

'__struct__' =>

'Elixir.Nostrum.Error.ApiError',

'__exception__' => true,

status_code => 403},

protocol => 'ElixirJason.Encoder',

description =>

>,

'__struct__' =>

'Elixir.Protocol.UndefinedError',

'__exception__' => true}

Callback modules = ['Elixir.Nostrum.Api.Ratelimiter']

Callback mode = state_functions

Stacktrace =

['ElixirJason','encode_to_iodata!',2,

[{file,"lib/jason.ex"},

{line,213},

```
{error_info,#{module => 'Elixir.Exception'}}},
{'Elixir.Nostrum.Api.Adapter',request,7,
[{file,"lib/nostrum/api/adapter.ex"},{line,36}]},
{'Elixir.Nostrum.Api.Ratelimiter',connected,3,
[{file,"lib/nostrum/api/ratelimiter.ex"},{line,514}]},
{gen_statem,loop_state_callback,11,[{file,"gen_statem.erl"},{line,3741}]},
{proc_lib,init_p_do_apply,3[{file,"proc_lib.erl"},{line,333}]}
```

```
Time-outs: {2,
[{{timeout,>},expired},
{{timeout,>},expired}]}
```

22:43:10.875 [error] Task #PID started from #PID terminating

```
(stop) exited in: :gen_statem.call(#PID, {:queue, %{params: [], body: %{embeds:
[%Nostrum.Struct.Embed{title: " Erro na Roleta Russa", type: nil, description:
%Nostrum.Error.ApiError{status_code: 403, response: %{code: 50013, message: "Missing
Permissions"}}, url: nil, timestamp: nil, color: 16711680, footer: nil, image: nil, thumbnail: nil, video:
nil, provider: nil, author: nil, fields: nil}}, headers: [{"content-type", "application/json"}], method:
:post, route: "/channels/1502805027633365112/messages"}}, :infinity)
```

(EXIT) an exception was raised:

(Protocol.UndefinedError) protocol Jason.Encoder not implemented for Nostrum.Error.ApiError (a struct), Jason.Encoder protocol must always be explicitly implemented.

If you own the struct, you can derive the implementation specifying which fields should be encoded to JSON:

```
@derive {Jason.Encoder, only: [....]}
```

```
defstruct ...
```

It is also possible to encode all fields, although this should be used carefully to avoid accidentally leaking private information when new fields are added:

```
@derive Jason.Encoder
```

```
defstruct ...
```

Finally, if you don't own the struct you want to encode to JSON, you may use Protocol.derive/3

placed outside of any module:

```
Protocol.derive(Jason.Encoder, NameOfTheStruct, only: [...])
```

```
Protocol.derive(Jason.Encoder, NameOfTheStruct)
```

. This protocol is implemented for: Any, Atom, BitString, Date, DateTime, Float, Integer, Jason.Fragment, Jason.OrderedObject, List, Map, NaiveDateTime, Nostrum.Struct.Component, Nostrum.Struct.Component.DefaultValue, Nostrum.Struct.Component.Option, Nostrum.Struct.Embed, Nostrum.Struct.Embed.Author, Nostrum.Struct.Embed.Field, Nostrum.Struct.Embed.Footer, Nostrum.Struct.Embed.Image, Nostrum.Struct.Embed.Provider, Nostrum.Struct.Embed.Thumbnail, Nostrum.Struct.Embed.Video, Nostrum.Struct.Guild.ScheduledEvent.EntityMetadata, Nostrum.Struct.Guild.UnavailableGuild, Nostrum.Struct.Message.Poll, Nostrum.Struct.Message.Poll.Answer, Nostrum.Struct.Message.Poll.MediaObject, Nostrum.Struct.Webhook, Time

Got value:

```
%Nostrum.Error.ApiError{  
  
  status_code: 403,  
  
  response: %{code: 50013, message: "Missing Permissions"}  
  
}
```

(jason 1.4.5) lib/jason.ex:213: Jason.encode_to_iodata!/2

(nostrum 0.10.4) lib/nostrum/api/adapter.ex:36: Nostrum.Api.Adapter.request/7

(nostrum 0.10.4) lib/nostrum/api/ratelimiter.ex:514: Nostrum.Api.Ratelimiter.connected/3

(stdlib 7.3) gen_statem.erl:3741: :gen_statem.loop_state_callback/11

(stdlib 7.3) proc_lib.erl:333: :proc_lib.init_p_do_apply/3

(stdlib 7.3) gen.erl:249: :gen.do_call/4

(stdlib 7.3) gen_statem.erl:3254: :gen_statem.call/3

(nostrum 0.10.4) lib/nostrum/api/message.ex:110: Nostrum.Api.Message.create/2

(davibot 0.1.0) deps/nostrum/lib/nostrum/consumer.ex:448: anonymous fn/1 in
Davibot.Consumer.handle_info/2

(elixir 1.19.5) lib/task/supervised.ex:105: Task.Supervised.invoke_mfa/2

Function: #Function

Args: []

IA:

Estou analisando esse erro. Parece que o problema está na comunicação com a API, especificamente com permissões e a codificação de dados.

Esse log de erro aponta para dois problemas distintos que estão ocorrendo simultaneamente: um erro de **permissão no Discord** e uma falha de **codificação no Elixir**.

1. O Erro de Codificação (Jason.Encoder)

O erro principal que causou o travamento (Protocol.UndefinedError) aconteceu porque você tentou enviar a struct `%Nostrum.Error.ApiError{}` diretamente dentro do campo `description` de um Embed.

Quando o Nostrum tenta enviar essa mensagem, ele utiliza a biblioteca **Jason** para converter os dados em JSON. Como a struct `Nostrum.Error.ApiError` não possui uma implementação do protocolo `Jason.Encoder`, o sistema falha ao tentar transformar esse objeto Elixir em texto para a API do Discord.

2. O Erro de Origem (Missing Permissions)

Dentro da struct que falhou na codificação, vemos a causa real do problema original:

- **Status Code:** 403
- **Discord Code:** 50013
- **Mensagem:** "Missing Permissions"

Isso significa que, antes do crash, o bot tentou realizar uma ação (provavelmente o banimento ou o envio da mensagem no canal) e o Discord negou porque o bot não possui as permissões necessárias (`BAN_MEMBERS` para banir ou `EMBED_LINKS` para enviar o log de erro).

Como Corrigir o Código

Você precisa garantir que o conteúdo enviado ao Embed seja uma **String**, e não a struct de erro completa. No seu arquivo `consumer.ex` (ou onde você trata o erro da "Roleta Russa"), altere a forma como você passa o erro para o Embed:

Código Problemático:

```
```elixir
```

## Isso causa o erro porque 'reason' é uma struct

```
Api.Message.create(channel_id, embeds: [%{description: reason}])
```

```
```
```

Código Corrigido:

```
```elixir
```

## Use inspect/1 para transformar a struct em uma string legível

```
Api.Message.create(channel_id, embeds: [%{
 title: " Erro na Roleta Russa",
 description: "Falha ao banir: #{inspect(reason)}",
 color: 0xFF0000
}])
```

```
```
```

Se quiser algo mais limpo, acesse apenas a mensagem da resposta:

```
```elixir  

description: "Erro: #{reason.response.message}"
```

```
```
```

Resumindo

1. **Causa Técnica:** Você tentou serializar uma struct de erro do Nostrum como JSON dentro de um Embed, o que não é suportado pelo codificador Jason.
2. **Causa Funcional:** O bot recebeu um erro 403 Forbidden do Discord. Ele não tem permissão

para banir membros ou para enviar embeds naquele canal específico.

3. **Solução:** Utilize `inspect(reason)` para converter o erro em texto antes de enviá-lo e verifique as permissões do cargo do bot no servidor do Discord.

Próximos Passos

- **Verificar Permissões:** No Discord, suba o cargo do bot para o topo da lista e garanta que ele tenha as permissões de "Banir Membros" e "Inserir Links".
- **Tratamento de Erros:** Revise a função `ban_member` para garantir que todos os retornos de erro (`{:error, reason}`) passem por um `inspect/1` antes de serem exibidos em mensagens de interface.
- **Log de Auditoria:** Verifique se o bot está tentando banir alguém com um cargo superior ao dele, o que também gera erro de permissão.

Usuário:

lib/davibot/commands.exs

```
defmodule Davibot.Commands do
```

```
@moduledoc """
```

```
Aqui é implementado os comando do davibot
```

```
"""
```

```
#alias Nostrum.Api.Message
```

```
alias Nostrum.Api
```

```
alias DaviBot.Store
```

```
@doc """
```

```
!calabreso - retona o gif do calma calabreso
```

```
"""
```

```
def calabreso(%{channel_id: cid}) do
```

```
  gif_url =
```

```
  "https://imgs.search.brave.com/7jfQ8lDvoNZdLt8966NScrKRAXLXCng7TLKro2Boifc/rs:fit:860:0:0:0/g:ce/aHR0cHM6Ly9tZWRRp/YS50ZW5vci5jb20v/TTRUNI9LSF9HV01B/QUFBTS9jYWxtYS1j/YWxh
```

YnJlc28tY2Fs/bWEtY2FsbWEtY2Fs/YWJyZXNvLmdpZg.gif"

embed = %Nostrum.Struct.Embed{

title: "Toma, calabreso",

image: %Nostrum.Struct.Embed.Image{url: gif_url},

color: 0x00ff00

}

Api.Message.create(cid, embeds: [embed])

end

def roleta_russa(msg) do

#IO.inspect(Api.Guild.members(msg.guild_id, limit: 1000))

#IO.inspect(Api.Guild.get(msg.guild_id))

#IO.inspect(Api.Guild.integrations(msg.guild_id))

#IO.inspect(bot_id = Api.Self.application_information())

with {:ok, n} <- parse_number(msg.content),

{:ok, guild} <- Api.Guild.get(msg.guild_id),

{:ok, members} <- Api.Guild.members(msg.guild_id, limit: 1000),

{:ok, bot_id} <- get_bot_id(),

filtered <- filter_members(members, guild.owner_id, bot_id),

true = n,

selected Enum.take(n),

:ok <- perform_bans(msg.guild_id, selected) do

IO.inspect("Roleta Russa: banned #{n} members in guild #{msg.guild_id}")

success_embed(n)

else

{:error, reason} -> error_embed(reason)

```
false -> error_embed("Not enough eligible members")
```

```
end
```

```
end
```

```
@doc """
```

```
Obtém o bot_id via Api.Self.application_information()
```

```
Retorna {:ok, bot_id} ou {:error, reason}
```

```
"""
```

```
def get_bot_id do
```

```
  with {:ok, %{bot: %{id: bot_id}}} <- Api.Self.application_information() do
```

```
    {:ok, bot_id}
```

```
  else
```

```
    _ -> {:error, "Falha ao obter ID do bot"}
```

```
  end
```

```
end
```

```
defp parse_number(content) do
```

```
  case Regex.run(~r/!(roleta[_\s]+russa) (\d+)/i, String.trim(content)) do
```

```
    [_, _, number_str] ->
```

```
    case Integer.parse(number_str) do
```

```
      {n, ""} when n >= 1 and n
```

```
        {:ok, n}
```

```
      _ ->
```

```
        {:error, "Número inválido. Deve ser inteiro entre 1 e 500."}
```

```
    end
```

```
    _ ->
```

```
      {:error, "Formato inválido. Use: !roleta-russa "}
```

end

end

def filter_members(members, owner_id, bot_id) do

members

```
> Enum.filter(&(&1.user_id != owner_id))
```

```
> Enum.filter(&(&1.user_id != bot_id))
```

end

defp perform_bans(guild_id, selected) do

Enum.reduce_while(selected, :ok, fn member, _acc ->

case ban_member(guild_id, member) do

:ok -> {:cont, :ok}

{:error, reason} -> {:halt, {:error, reason}}

end

end)

end

defp ban_member(guild_id, member) do

Ajustado para retornar :ok puro conforme o padrão da API do Nostrum

case Api.Guild.ban_member(guild_id, member.user_id, 0, "roleta russa do davi britto") do

{:ok} ->

IO.inspect("Banned user #{member.user_id}")

:ok

{:error, reason} ->

```
IO.inspect("Failed to ban #{member.user_id}: #{inspect(reason)}")
```

```
{:error, reason}
```

```
end
```

```
end
```

```
defp success_embed(n) do
```

```
  %Nostrum.Struct.Embed{
```

```
    title: " Roleta Russa!",
```

```
    description: " Banidos #{n} membros aleatórios com sucesso!",
```

```
    color: 0xFF4444
```

```
  }
```

```
end
```

```
defp error_embed(reason) do
```

```
  %Nostrum.Struct.Embed{
```

```
    title: " Erro na Roleta Russa",
```

```
    description: reason,
```

```
    color: 0xFF0000
```

```
  }
```

```
end
```

```
end
```

Esse é o meu código atual, especifique aonde

IA:

Estou revisando o código.

O erro está localizado na função `error_embed/1`. O problema ocorre porque, quando o `ban_member` falha, a variável `reason` contém uma **struct** (`%Nostrum.Error.ApiError{}`), e você está

tentando passar essa struct diretamente para o campo description do Embed.

Para corrigir, você deve garantir que a description receba uma **String**.

Aqui estão as alterações necessárias no seu arquivo lib/davibot/commands.exs:

1. Corrigindo a função error_embed

Altere a função para usar inspect/1, que converterá qualquer struct ou erro em texto legível para o JSON:

```
```elixir

defp error_embed(reason) do

 %Nostrum.Struct.Embed{

 title: " Erro na Roleta Russa",
```

## O inspect transforma a struct de erro em String, evitando o erro do Jason.Encoder

```
 description: "Falha na operação: #{inspect(reason)}",

 color: 0xFF0000

 }

end

```
```

2. Ajuste opcional na perform_bans (Melhoria de legibilidade)

Se você quiser que o erro no Discord seja mais amigável (exibindo apenas a mensagem "Missing Permissions" em vez da struct técnica completa), você pode tratar o erro logo após o case na função ban_member:

```
```elixir

defp ban_member(guild_id, member) do

 case Api.Guild.ban_member(guild_id, member.user_id, 0, "roleta russa do davi britto") do
```

```

{:ok} ->

IO.inspect("Banned user #{member.user_id}")

:ok

{:error, %Nostrum.Error.ApiError{response: %{message: msg}}} ->

IO.inspect("Failed to ban #{member.user_id}: #{msg}")

{:error, msg} # Retorna apenas a string da mensagem de erro

{:error, reason} ->

{:error, inspect(reason)}

end

end

...

```

## Resumindo o motivo do erro:

O Discord exige que o campo `description` seja uma string. Quando ocorre o erro de permissão (403), o Nostrum retorna uma estrutura complexa de dados. Ao tentar enviar essa estrutura "viva" para o Discord, o codificador de JSON (Jason) trava porque não sabe como converter uma struct de erro do Nostrum em texto.

### Nota sobre Permissões:

Além do erro de código, o log confirmou que o seu bot está recebendo **"Missing Permissions"**. Verifique se:

1. O cargo do bot tem a permissão **Banir Membros**.
2. O cargo do bot está **acima** dos cargos dos membros que ele está tentando banir na lista de cargos do servidor.

## Usuário:

```
"Failed to ban 519171013666275339: %Nostrum.Error.ApiError{status_code: 403, response:
%{code: 50013, message: \"Missing Permissions\"}}"
```

estou tentando banir mas não dá. gerei uma url que permite o banimento de membros

## IA:

Estou analisando o erro que você recebeu. É interessante que você tenha gerado uma URL específica para essa ação.

O erro Missing Permissions (Código 50013) no Discord acontece quando o bot tenta executar uma ação (como banir um membro), mas o cargo dele não tem a permissão necessária ou ele está abaixo do cargo do alvo na hierarquia.

Se você gerou uma URL de convite para dar as permissões corretas ao bot, você precisa garantir que ele seja adicionado com a permissão **Administrador** (ou, no mínimo, **Banir Membros**).

## Como resolver agora:

1. **Hierarquia de Cargos:** No Discord, vá em Configurações do Servidor > Cargos. Arraste o cargo do seu bot para o **topo da lista**. O bot nunca consegue banir alguém que tenha um cargo acima do dele.
2. **Permissões da URL:** Para garantir que o bot tenha tudo o que precisa, a URL de convite deve ter o parâmetro permissions=4 (Banir) ou permissions=8 (Administrador).
3. **Uso no Código:** O erro que você viu antes também indicava que você estava tentando enviar a "struct" do erro diretamente no Embed. No seu código, certifique-se de usar inspect(reason) para exibir o erro como texto, como conversamos anteriormente.

## Resumindo

- **Erro 403 / 50013:** O bot não tem poder suficiente no servidor.
- **Solução:** Suba o cargo do bot na hierarquia e verifique se a permissão "Banir Membros" está ativa para ele.

**Se precisar de ajuda para ajustar a função que gera ou trata esse erro no seu código Elixir, é só me mandar o trecho!**

## Próximos passos

- [ ] Subir o cargo do bot no Discord.
- [ ] Testar o comando novamente em um usuário que não seja o dono do servidor.



- [ ] Verificar se o reason no error\_embed está com inspect.

## Usuário:

pesquise usando os links abaixo para me explicar a struct member e a função member e a função get\_avatar\_url

[https://hexdocs.pm/nostrum/Nostrum.Struct.Guild.Member.html#avatar\\_url/3](https://hexdocs.pm/nostrum/Nostrum.Struct.Guild.Member.html#avatar_url/3)

<https://hexdocs.pm/nostrum/Nostrum.Struct.Guild.Member.html#content>

<https://hexdocs.pm/nostrum/Nostrum.Api.Guild.html#member/2>

## IA:

Estou analisando os links para te explicar esses termos.

Abaixo, explico os detalhes técnicos solicitados com base na documentação da biblioteca

**Nostrum:**

### 1. Nostrum.Struct.Guild.Member (A Struct)

Esta struct representa um membro de um servidor (guild) no Discord. Ela contém informações específicas da relação do usuário com aquele servidor.

#### • Campos principais:

- user\_id: O ID do usuário.
- nick: O apelido (nickname) do usuário no servidor (pode ser nil).
- roles: Uma lista de IDs (Snowflakes) dos cargos que o membro possui.
- joined\_at: A data e hora em que o membro entrou no servidor.
- premium\_since: A data em que o membro começou a impulsionar (boost) o servidor (se aplicável).
- deaf e mute: Booleanos indicando se o usuário está com ensurdecimento ou silenciamento de servidor.
- avatar: O hash do avatar específico do servidor (caso o usuário tenha um avatar diferente do global para aquele servidor).

## 2. Nostrum.Api.Guild.member/2 (A Função)

Esta função é utilizada para buscar os dados de um membro específico através da API do Discord.

- **Assinatura:** `member(guild_id, user_id)`

- **Argumentos:**

1. `guild_id`: O ID do servidor.
2. `user_id`: O ID do usuário que você deseja buscar.

- **Retorno:**

- `{:ok, Nostrum.Struct.Guild.Member.t()}` em caso de sucesso.
- `{:error, reason}` em caso de falha (ex: membro não encontrado ou falta de permissão).

## 3. Nostrum.Struct.Guild.Member.avatar\_url/3 (Obtendo o Avatar)

Diferente do avatar global do usuário, os membros podem ter avatares específicos por servidor. A função `avatar_url/3` (ou similares em versões da struct) ajuda a construir o link dessa imagem.

- **Funcionamento:**

- Ela verifica se o membro possui um avatar definido na struct `Member`.
- Se possuir, ela gera a URL apontando para os servidores de CDN do Discord com o hash do avatar do membro.
- Se **não** possuir um avatar de servidor, você deve recorrer à função de avatar da struct `User` (`Nostrum.Struct.User.avatar_url(user)`), que retornará o avatar global.

- **Argumentos comuns:**

1. A própria struct `member`.
2. O formato da imagem (ex: "png", "webp", "jpg").
3. O tamanho da imagem (ex: 128, 256, 512).

## Resumo Comparativo

Recurso	Tipo	Função Principal
Member (Struct)	Dados	Armazena cargos, apelido e data de entrada.

member/2 (API)	Ação	Busca um membro específico no servidor via rede.
avatar_url/3	Helper	Gera o link da imagem do perfil (priorizando a do servidor).

**Dica técnica:** Sempre verifique se o bot tem a intenção (intent) GUILD\_MEMBERS habilitada no portal do desenvolvedor do Discord e no seu código para conseguir listar ou buscar membros com sucesso.

## Usuário:

então nessa função, o correto seria usar `Nostrum.Api.Guild.member/2` (A Função)?

```
defp get_user_info(guild_id, user_id) do
```

## Tenta pegar do cache primeiro, depois da API

```
member = %Nostrum.Struct.Guild.member{user_id: user_id}
```

```
case %Nostrum.Struct.Guild.member{user_id: user_id} do
```

```
{:ok, member} -> {:ok, member}
```

```
{:error, _} -> :error
```

```
end
```

```
end
```

## IA:

Estou verificando essa informação.

Sim, exatamente. O seu código atual tem um erro conceitual: você está tentando usar um ``case`` em cima de uma literal de **Struct** (`%Nostrum.Struct.Guild.Member{...}`), o que no Elixir apenas cria um mapa de dados na memória e nunca retornará `{:ok, member}` ou `{:error, _}`.

Para buscar as informações de um membro no servidor, você deve usar a função ``Nostrum.Api.Guild.member/2``.

Abaixo, apresento como seria a implementação correta dessa função, incluindo uma boa prática: tentar buscar no **Cache** primeiro (para economizar recursos e evitar limites de taxa/rate limits) e, caso não encontre, buscar na **API**.

## O que estava errado no seu código original:

1. **Struct não é função:** Ao fazer `%Nostrum.Struct.Guild.Member{}`, você está apenas criando um "molde" de dados. Isso não executa nenhuma ação de busca no servidor do Discord.
2. **Case inválido:** Como a Struct é apenas um dado, o case nela sempre falharia ao tentar encontrar o padrão `{:ok, member}`.
3. **Lowercase:** Em Elixir, módulos e Structs começam com letra maiúscula (Member, não member).

## Resumindo as funções:

- ``Nostrum.Cache.MemberCache.get/2``: Busca instantânea na memória RAM do seu bot (muito rápido, não gasta internet).
- ``Nostrum.Api.Guild.member/2``: Faz uma requisição HTTP para o Discord (mais lento, sujeito a erros de rede e permissão).

## Próximos passos:

- Certifique-se de que o módulo `Nostrum.Api` esteja devidamente configurado com o seu token.
- Verifique se o bot tem as **Intents** de `GUILD_MEMBERS` ativadas no Portal do Desenvolvedor do Discord, caso contrário, o Cache poderá ficar vazio.

## Usuário:

```
@doc """
```

```
 Constrói URL do avatar (personalizado ou padrão)
```

```
"""
```

```
defp build_avatar_url({:ok, user}, size) do
```

```
 user_id_str = to_string(user.id)
```

```
avatar_url =
```

```
cond do
```

## Usuário tem avatar personalizado

```
user.avatar != nil and user.avatar != "" ->
```

```
"https://cdn.discordapp.com/avatars/#{user_id_str}/#{user.avatar}.png?size=#{size}"
```

## Usa avatar padrão do Discord (baseado no discriminator)

```
true ->
```

```
disc_num = String.to_integer(user.discriminator) |> rem(5)
```

```
"https://cdn.discordapp.com/embed/avatars/#{disc_num}.png"
```

```
end
```

```
{:ok, avatar_url, user}
```

```
end
```

```
defp build_avatar_url(:error, _size), do: :error
```

vamos aproveitar e adaptar essa função porque ela vai receber o member de `get_user_info`. E aproveitar e usar a função `avatar` da própria biblioteca ao invés de criar uma string do zero

## IA:

Estou analisando o código que você enviou.

Excelente ideia. Utilizar as funções nativas da biblioteca é a melhor prática, pois o **Nostrum** já lida com as complexidades do CDN do Discord (como extensões de arquivo e avatares padrão) automaticamente.

Note que para o avatar do membro (específico do servidor), a biblioteca exige o `guild_id`.

Aqui está a refatoração sugerida para o seu módulo:

## Refatoração da Função

```
``elixir
```

```
@doc """
```

Constrói URL do avatar priorizando o avatar do servidor (member),

caindo para o avatar global (user) se necessário.

```
"""
```

```
defp build_avatar_url(%Nostrum.Struct.Guild.Member{user: user} = member, guild_id, size) do
```

## Tenta obter o avatar específico do servidor primeiro

```
avatar_url =
```

```
case Nostrum.Struct.Guild.Member.avatar_url(member, guild_id, size) do
```

```
nil ->
```

## Se não houver avatar de servidor, busca o avatar global do usuário

```
Nostrum.Struct.User.avatar_url(user, size)
```

```
url ->
```

```
url
```

```
end
```

```
{:ok, avatar_url, user}
```

```
end
```

```
defp build_avatar_url(_, _, _), do: :error
```

```
```
```

O que mudou e por que é melhor:

1. **Nostrum.Struct.Guild.Member.avatar_url/3:** Essa função já verifica internamente se o membro possui um avatar de servidor. Se não possuir, ela retorna nil, permitindo o fallback.
2. **Nostrum.Struct.User.avatar_url/2:** Quando chamada para o usuário, ela já lida com a lógica que você tinha manualmente (verificar se o hash existe ou retornar o avatar padrão do Discord baseado no discriminador).
3. **Manutenibilidade:** Se o Discord mudar o domínio do CDN (como já aconteceu de cdn.discordapp.com para outros formatos no passado), você não precisará atualizar seu código, apenas a biblioteca.

Resumo Técnico

- **Prioridade:** Avatar do Servidor (Member) > Avatar Global (User) > Avatar Padrão (automático pelo Nostrum).
- **Parâmetros:** Agora você passa a struct completa e o ID da guilda.
- **Retorno:** Mantivemos o seu padrão de retorno `{:ok, url, user}` para não quebrar as funções que chamam esta.

Dica: No seu `get_user_info`, certifique-se de que a struct member retornada já venha com o campo `user` preenchido (o que é o padrão do cache/API do Nostrum). Se não vier, você precisará fazer o preload do usuário.

Usuário:

```
def build_avatar_url(%Nostrum.Struct.Guild.Member{user: %Nostrum.Struct.User{} = user} =
member, guild_id, size) do

  if member.avatar do

    url = Nostrum.Struct.Guild.Member.avatar_url(member, guild_id)|> Enum.join("?size=#{size}")

    {:ok, url, user}

  else

    url = Nostrum.Struct.User.avatar_url(user)|>Enum.join("?size=#{size}")

    {:ok, url, user}

  end

end
```

nesse caso, eu tentei pegar a url que foi criada e após URL adicionar o tamanho dela

IA:

Estou analisando o código que você enviou. A lógica parece estar focada em construir URLs de avatar com um parâmetro de tamanho.

O erro na sua tentativa de concatenação (`|> Enum.join("?size=#{size}")`) acontece porque as funções `avatar_url` do Nostrum não retornam uma lista, mas sim uma **String** (ou nil). O `Enum.join` espera uma lista, por isso a falha.

Além disso, as funções nativas do Nostrum já aceitam o formato e o tamanho como argumentos, o que dispensa a necessidade de manipular a String manualmente.

Aqui está a forma correta e idiomática de implementar isso usando as capacidades da biblioteca:

Código Corrigido

```
```elixir
```

```
defp build_avatar_url(%Nostrum.Struct.Guild.Member{user: user} = member, guild_id, size) do
```

## O Nostrum já gera a URL completa com o tamanho se passarmos os argumentos corretamente

```
avatar_url =
```

```
if member.avatar do
```

```
avatar_url(member, guild_id, image_format \\
"webp", size \\
1024)
```

```
Nostrum.Struct.Guild.Member.avatar_url(member, guild_id, "png", size)
```

```
else
```

```
avatar_url(user, image_format \\
"webp", size \\
1024)
```



```
Nostrum.Struct.User.avatar_url(user, "png", size)
```

```
end
```

```
{:ok, avatar_url, user}
```

```
end
```

```
...
```

## Por que usar assim?

1. **Tipagem Correta:** As funções `avatar_url` da biblioteca retornam a URL pronta. Ao passar `size` como o quarto argumento (no caso de `Member`) ou segundo (no caso de `User`), a biblioteca já anexa o `?size=N` corretamente seguindo os padrões da API do Discord.
2. **Tratamento de Extensões:** O Discord suporta múltiplos formatos. Usando a função nativa, você garante que, se um avatar for um GIF (animado), a biblioteca saiba lidar com a extensão correta se você permitir.
3. **Segurança:** Manipular Strings de URL manualmente pode levar a erros de interrogação dupla (`??size`) ou parâmetros mal formados. A biblioteca abstrai isso.

## Resumindo

- **Não use `Enum.join`** em Strings; use a interpolação `#{url}?size=#{size}` se realmente precisar manipular o texto.
- **Prefira passar o tamanho como argumento** diretamente na função do Nostrum, como mostrado no exemplo acima.

### Próximo passo:

Verifique se você está importando corretamente os módulos das Structs no topo do seu arquivo para evitar erros de compilação com as referências `%Nostrum.Struct.Guild.Member{}`.

## Usuário:

então como posso dar um concat nessa string?

## IA:

Estou buscando isso. Manipular strings é um dos usos mais comuns de funções em programação.

Para concatenar o parâmetro de tamanho à URL em Elixir, o método correto é utilizar a **interpolação de strings** com o operador `#{}`.

Como as funções `avatar_url` do `Nostrum` retornam uma string (e não uma lista), você não deve usar `Enum.join`. O código corrigido para o seu caso seria:

```
```elixir

def build_avatar_url(%Nostrum.Struct.Guild.Member{user: %Nostrum.Struct.User{} = user} =
member, guild_id, size) do

if member.avatar do
```

Concatenação manual via interpolação

```
base_url = Nostrum.Struct.Guild.Member.avatar_url(member, guild_id)

url = "#{base_url}?size=#{size}"

{:ok, url, user}

else
```

Concatenação manual via interpolação

```
base_url = Nostrum.Struct.User.avatar_url(user)

url = "#{base_url}?size=#{size}"

{:ok, url, user}

end

end

```
```

## Melhor Abordagem

Embora a interpolação funcione, a biblioteca **Nostrum** já suporta o tamanho como argumento

**nativo.** Isso é mais seguro pois a biblioteca cuida da formatação correta da query string:

```
``elixir
```

```
def build_avatar_url(%Nostrum.Struct.Guild.Member{user: user} = member, guild_id, size) do
```

```
url =
```

```
if member.avatar do
```

## Passando o tamanho diretamente (formato padrão é webp ou png)

```
Nostrum.Struct.Guild.Member.avatar_url(member, guild_id, "png", size)
```

```
else
```

## Passando o tamanho diretamente

```
Nostrum.Struct.User.avatar_url(user, "png", size)
```

```
end
```

```
{:ok, url, user}
```

```
end
```

```
...
```

## Resumo

1. **Interpolação:** Use "#{string\_a}#{string\_b}" para concatenar.
2. **Nativo:** Prefira passar o size como argumento da função do Nostrum para evitar erros de formatação na URL.

### Próximos passos:

- Verifique se o guild\_id que você está passando é o ID numérico correto.
- Se o avatar for opcional, garanta que a função trate casos onde a URL possa retornar nil.

## Usuário:

mas gostaria de usar usando comandos pipe com pattern mating, se possível, se não tiver como, sem problemas

na função *build\_avatar\_url* sem usar o arg de size na chamada de *Nostrum.Struct.Guild.Member.avatar\_url*

## IA:

Estou analisando a sua solicitação. A abordagem com pipe e pattern matching pode ser interessante para essa função.

É perfeitamente possível utilizar o operador pipe (*|>*) e **Pattern Matching** para deixar o código bem idiomático ("estilo Elixir").

Para concatenar manualmente sem usar o argumento nativo de size, utilizamos o operador *<>* dentro de uma função anônima no final do pipe.

## Refatoração com Pipes e Pattern Matching

```
```elixir
```

Caso 1: Membro não possui avatar no servidor (avatar: nil)

Faz o matching diretamente no cabeçalho e busca o avatar global do usuário

```
defp build_avatar_url(%Nostrum.Struct.Guild.Member{avatar: nil, user: user}, _guild_id, size) do
  user
```

```
> Nostrum.Struct.User.avatar_url()
```

```
> (fn url -> url <> "?size=#{size}" end).()
```

```
> then(&{:ok, &1, user})
```

end

Caso 2: Membro possui avatar no servidor

O matching anterior falha, então este aqui processa o avatar da guilda

```
defp build_avatar_url(%Nostrum.Struct.Guild.Member{user: user} = member, guild_id, size) do
  member
```

```
> Nostrum.Struct.Guild.Member.avatar_url(guild_id)
```

```
> (fn url -> url <> "?size=#{size}" end).()
```

```
> then(&{:ok, &1, user})
```

end

Caso 3: Fallback de erro

```
defp build_avatar_url(_, _, _), do: :error
```

```
...
```

O que foi aplicado aqui:

1. **Pattern Matching no Argumento:** O primeiro defp captura especificamente quando avatar: nil. Isso elimina a necessidade de um if/else dentro da função, seguindo a filosofia de Elixir.
2. **Operador Pipe (`|>`):** O fluxo de dados é linear. Pegamos a struct, transformamos em URL e depois modificamos a string.
3. **Concatenação Manual:** Como você pediu para não usar o argumento size da função nativa, usamos url <> "?size=#{size}". Note que usamos (). para invocar a função anônima dentro do pipe.
4. **`then/2`:** Usei esta função (disponível a partir do Elixir 1.12) para empacotar o resultado final na tupla {:ok, url, user} mantendo o encadeamento dos pipes.

Resumo

- **Vantagem:** O código fica muito mais legível e fácil de testar para casos específicos.
- **Dica:** O uso de `avatar: nil` no matching é a forma mais eficiente de decidir qual URL buscar antes mesmo de entrar no corpo da função.

Próximos passos:

- Verifique se a versão do seu Elixir é 1.12 ou superior para usar o `then`. Caso seja anterior, você pode finalizar o pipe apenas com a string e montar a tupla na linha seguinte.